

SoulSense AI - Unified Project Architecture

Single Project Structure

This is a **unified full-stack project** where frontend and backend are properly integrated and aligned as one cohesive application.

soulsense-ai/	
└─ frontend/	→ React WebApp (your UI requirements)
│ └─ public/	
│ │ └─ index.html	
│ │ └─ manifest.json	
│ └─ src/	
│ │ └─ components/	
│ │ └─ PersonaSelector.jsx	→ Persona Selection (Dr. Sarah, Alex, Marcus, Maya)
│ │ └─ ChatScreen.jsx	→ Main Replika-style chat UI
│ │ └─ TopNavBar.jsx	→ Navigation (Chat Memory Diary Profile)
│ │ └─ MessageBubble.jsx	→ Chat bubbles (AI left, User right)
│ │ └─ InputBar.jsx	→ Typing bar (Text + Emoji + Mic + Attachments)
│ │ └─ SessionSummary.jsx	→ End of Session Summary screen
│ │ └─ MemoryTimeline.jsx	→ Memory section component
│ │ └─ DiaryScreen.jsx	→ Diary section component
│ │ └─ ProfileScreen.jsx	→ Profile section component
│ │ └─ TypingIndicator.jsx	→ AI typing animation
│ │ └─ assets/	
│ │ │ └─ persona_avatars/	→ Avatars for the 4 personas
│ │ │ └─ backgrounds/	→ Background gradient images
│ │ │ └─ icons/	→ UI icons
│ │ └─ services/	
│ │ │ └─ api.js	→ Connects to backend endpoints
│ │ │ └─ chatService.js	→ Calls /api/chat endpoints
│ │ │ └─ personaService.js	→ Calls /api/personas endpoints
│ │ │ └─ sessionService.js	→ Calls /api/sessions endpoints
│ │ └─ hooks/	
│ │ │ └─ useChat.js	→ Frontend chat state management
│ │ │ └─ usePersona.js	→ Frontend persona state
│ │ │ └─ useSession.js	→ Frontend session state
│ │ └─ App.jsx	→ Routing: PersonaSelector → ChatScreen → Summary
Summary	
│ └─ index.js	→ React entry point
│ └─ index.css	→ Global CSS styles
│ └─ package.json	→ Frontend dependencies
│ └─ .env	→ Frontend environment variables
└─ backend/	→ FastAPI Backend (your requirements)
│ └─ app/	
│ │ └─ __init__.py	
│ │ └─ main.py	→ FastAPI app entry point
│ │ └─ components/	
│ │ │ └─ __init__.py	
│ │ │ └─ persona_selector.py	→ API serving available personas

└─ chat_interface.py	→ Main chat API endpoint
└─ session_summary.py	→ API for session summary data
└─ routers/	
└─ __init__.py	
└─ personas.py	→ /api/personas/* endpoints
└─ chat.py	→ /api/chat/* endpoints
└─ sessions.py	→ /api/sessions/* endpoints
└─ health.py	→ /api/health endpoint
└─ models/	
└─ __init__.py	
└─ user.py	→ Pydantic user models
└─ conversation.py	→ Pydantic conversation models
└─ persona.py	→ Pydantic persona models
└─ session.py	→ Pydantic session models
└─ database/	
└─ __init__.py	
└─ connection.py	→ Database setup
└─ session_logger.py	→ Persist conversations
└─ user_manager.py	→ User profiles
└─ pipeline/	
└─ __init__.py	
└─ controller.py	→ Master conversation controller
└─ conversation_manager.py	→ Multi-turn dialogue handler
└─ safety_monitor.py	→ Crisis detection & intervention
└─ models/	→ AI/ML Models
└─ __init__.py	
└─ emotion/	
└─ __init__.py	
└─ text_emotion.py	→ RoBERTa emotion classifier
└─ clinical/	
└─ __init__.py	
└─ invisible_assessment.py	→ PHQ-9 / GAD-7 extraction
└─ dialogue/	
└─ __init__.py	
└─ dialogue_manager.py	→ LLM-based response generator
└─ persona/	
└─ __init__.py	
└─ personality_engine.py	→ Persona conditioning
└─ engine/	
└─ __init__.py	
└─ adaptive_questioning.py	→ Dynamic interventions
└─ suggestion_engine.py	→ CBT, DBT, ACT suggestions
└─ data/	
└─ __init__.py	
└─ user_logs/	→ Conversation logs (for memory)
└─ clinical_progress/	→ PHQ-9 / GAD-7 progression
└─ therapeutic_outcomes/	→ Long-term improvement

requirements.txt	→ Backend dependencies
.env	→ Backend environment variables
└ shared/	→ Shared configurations
docker-compose.yml	→ Full stack orchestration
nginx.conf	→ Reverse proxy config
database/	
init.sql	→ Database initialization
migrations/	→ Database migrations
└ scripts/	
setup.sh	→ Full project setup
start-dev.sh	→ Start development servers
build.sh	→ Build for production
deploy.sh	→ Production deployment
└ tests/	
frontend/	→ Frontend tests
components/	→ Component tests
integration/	→ Frontend integration tests
backend/	→ Backend tests
unit/	→ Unit tests
integration/	→ API integration tests
└ docs/	
API.md	→ API documentation
SETUP.md	→ Project setup guide
ARCHITECTURE.md	→ Architecture overview
└ .env.example	→ Environment template
└ .gitignore	→ Git ignore rules
└ README.md	→ Project documentation
└ package.json	→ Root package.json for scripts

Frontend-Backend Integration

API Endpoints Alignment

Frontend Components ↔ Backend Endpoints

javascript

```
// Frontend: PersonaSelector.jsx
import { personaService } from '../services/personaService';

const PersonaSelector = () => {
  const [personas, setPersonas] = useState([]);

  useEffect(() => {
    // Calls backend: /api/personas/list
    personaService.getAvailablePersonas()
      .then(setPersonas);
  }, []);

  return (
    <div className="persona-grid">
      {personas.map(persona => (
        <PersonaCard
          key={persona.id}
          persona={persona}
          onSelect={() => handlePersonaSelect(persona.id)}
        />
      ))}
    </div>
  );
};
```

python

```
# Backend: app/components/persona_selector.py
from fastapi import APIRouter
from typing import List

router = APIRouter(prefix="/api/personas")

@router.get("/list")
async def get_available_personas() -> List[PersonaResponse]:
    """Returns the 4 personas: Dr. Sarah, Alex, Marcus, Maya"""
    return [
        PersonaResponse(
            id="dr-sarah",
            name="Dr. Sarah Chen",
            role="Clinical Psychologist",
            specialty="CBT Therapy",
            avatar_url="/assets/persona_avatars/dr_sarah.jpg",
            description="Warm, professional therapist specializing in CBT"
        ),
        PersonaResponse(
            id="alex",
            name="Alex Rodriguez",
            role="Peer Counselor",
            specialty="Relatability",
            avatar_url="/assets/persona_avatars/alex.jpg",
            description="Friendly peer who understands your struggles"
        ),
        PersonaResponse(
            id="marcus",
            name="Marcus Thompson",
            role="Life Coach",
            specialty="Goal Achievement",
            avatar_url="/assets/persona_avatars/marcus.jpg",
            description="Motivational coach focused on personal growth"
        ),
        PersonaResponse(
            id="maya",
            name="Maya Patel",
            role="Mindfulness Expert",
            specialty="Meditation & Mindfulness",
            avatar_url="/assets/persona_avatars/maya.jpg",
            description="Calm guide for mindfulness and meditation"
        )
    ]
```



```

// Frontend: ChatScreen.jsx
import { chatService } from '../services/chatService';

const ChatScreen = () => {
  const [messages, setMessages] = useState([]);
  const [currentPersona, setCurrentPersona] = useState(null);

  const sendMessage = async (userMessage) => {
    // Add user message to UI immediately
    const newMessage = {
      id: Date.now(),
      content: userMessage,
      sender: 'user',
      timestamp: new Date()
    };
    setMessages(prev => [...prev, newMessage]);

    // Call backend chat endpoint
    const response = await chatService.sendMessage({
      message: userMessage,
      persona_id: currentPersona.id,
      conversation_id: currentConversationId
    });

    // Add AI response to UI
    setMessages(prev => [...prev, {
      id: response.message_id,
      content: response.content,
      sender: 'ai',
      timestamp: new Date(response.timestamp),
      emotion_detected: response.emotion_analysis
    }]);
  };

  return (
    <div className="chat-container">
      <TopNavBar />
      <div className="messages-container">
        {messages.map(message => (
          <MessageBubble
            key={message.id}
            message={message}
            isAI={message.sender === 'ai'}
          />
        ))}
      </div>
    </div>
  );
}

```

```

        <InputBar onSendMessage={sendMessage} />
    </div>
);
};

```

python

```

# Backend: app/components/chat_interface.py
from fastapi import APIRouter, Depends
from app.pipeline.controller import ConversationController
from app.models.persona.personality_engine import PersonalityEngine

router = APIRouter(prefix="/api/chat")

@router.post("/message")
async def send_message(request: ChatMessageRequest) -> ChatMessageResponse:
    """Main chat endpoint - processes user message and returns AI response"""

    # Initialize conversation controller
    controller = ConversationController()

    # Process the message through the pipeline
    response = await controller.process_message(
        user_message=request.message,
        persona_id=request.persona_id,
        conversation_id=request.conversation_id,
        user_id=request.user_id
    )

    return ChatMessageResponse(
        message_id=response.message_id,
        content=response.content,
        timestamp=response.timestamp,
        emotion_analysis=response.emotion_analysis,
        clinical_indicators=response.clinical_indicators,
        session_progress=response.session_progress
    )

```

Session Summary Integration

javascript

// Frontend: SessionSummary.jsx

```
import { sessionService } from '../services/sessionService';
```

```
const SessionSummary = ({ conversationId }) => {  
  const [summary, setSummary] = useState(null);
```

```
  useEffect(() => {  
    // Calls backend: /api/sessions/summary  
    sessionService.getSessionSummary(conversationId)  
      .then(setSummary);  
  }, [conversationId]);
```

```
  return (  
    <div className="session-summary">  
      <h2>Session Summary</h2>  
      <div className="mood-analysis">  
        <h3>Mood Analysis</h3>  
        <p>Primary Emotion: {summary?.emotion_analysis?.primary_emotion}</p>  
        <p>Mood Score: {summary?.mood_score}/10</p>  
      </div>  
      <div className="key-insights">  
        <h3>Key Insights</h3>  
        <ul>  
          {summary?.insights?.map((insight, index) => (  
            <li key={index}>{insight}</li>  
          ))}  
        </ul>  
      </div>  
      <div className="next-steps">  
        <h3>Recommended Next Steps</h3>  
        <ul>  
          {summary?.recommendations?.map((rec, index) => (  
            <li key={index}>{rec}</li>  
          ))}  
        </ul>  
      </div>  
    </div>  
  );  
};
```

python

```
# Backend: app/components/session_summary.py
from fastapi import APIRouter
from app.models.clinical.invisible_assessment import InvisibleAssessment
from app.engine.suggestion_engine import SuggestionEngine

router = APIRouter(prefix="/api/sessions")

@router.get("/summary/{conversation_id}")
async def get_session_summary(conversation_id: str) -> SessionSummaryResponse:
    """Generate session summary with insights and recommendations"""

    # Get conversation data
    conversation = await get_conversation(conversation_id)

    # Analyze emotions throughout session
    emotion_analyzer = EmotionAnalyzer()
    emotion_analysis = emotion_analyzer.analyze_session(conversation.messages)

    # Extract clinical insights
    clinical_analyzer = InvisibleAssessment()
    clinical_insights = clinical_analyzer.extract_insights(conversation.messages)

    # Generate recommendations
    suggestion_engine = SuggestionEngine()
    recommendations = suggestion_engine.generate_session_recommendations(
        emotion_analysis, clinical_insights
    )

    return SessionSummaryResponse(
        conversation_id=conversation_id,
        session_duration=conversation.duration_minutes,
        message_count=len(conversation.messages),
        emotion_analysis=emotion_analysis,
        mood_score=emotion_analysis.overall_mood_score,
        insights=clinical_insights.key_insights,
        recommendations=recommendations.next_steps,
        phq9_indicators=clinical_insights.phq9_score,
        gad7_indicators=clinical_insights.gad7_score
    )
```

Single Project Setup

Development Setup

```
bash
```

```
# Clone the unified project
```

```
git clone https://github.com/yourusername/soulsense-ai.git
```

```
cd soulsense-ai
```

```
# Setup script handles both frontend and backend
```

```
chmod +x scripts/setup.sh
```

```
./scripts/setup.sh
```

```
# Start development servers (both frontend and backend)
```

```
./scripts/start-dev.sh
```

Production Deployment

```
bash
```

```
# Build both frontend and backend
```

```
./scripts/build.sh
```

```
# Deploy as single application
```

```
./scripts/deploy.sh
```

Docker Compose (Single Stack)

yaml

```
# shared/docker-compose.yml
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      - REACT_APP_API_URL=http://localhost:8000
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/soulsense
    depends_on:
      - db

  db:
    image: postgres:14
    environment:
      - POSTGRES_DB=soulsense
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

✅ Does This Match Your Needs?

- ✓ **Single unified project** - Not separate projects
- ✓ **Frontend components exactly as you specified:**

- PersonaSelector.jsx (Dr. Sarah, Alex, Marcus, Maya)
- ChatScreen.jsx (Replika-style UI)
- TopNavBar.jsx (Chat | Memory | Diary | Profile)
- MessageBubble.jsx (AI left, User right)
- InputBar.jsx (Text + Emoji + Mic + Attachments)

- SessionSummary.jsx (End of session insights)

✓ **Backend components exactly as you specified:**

- persona_selector.py (API serving personas)
- chat_interface.py (Main chat API)
- session_summary.py (Session summary API)
- All your pipeline, models, engine, and data structure

✓ **Proper alignment:**

- Frontend services call exact backend endpoints
- Shared data models between frontend and backend
- Unified development and deployment process
- Single project repository with integrated architecture

This is now a **single cohesive project** where frontend and backend work together seamlessly, exactly matching your requirements!